

Hvilket af nedenstående svar gælder for følgende rekursionsligning?

$$T(n) = 2 \cdot T(n/4) + n^2$$

☐ $T(n) = \Theta(1)$

☐ $T(n) = \Theta(\log n)$

☐ $T(n) = \Theta(n^{\log_4 3})$

☐ $T(n) = \Theta(n)$

☐ $T(n) = \Theta(n \log n)$

☐ $T(n) = \Theta(n^{\log_3 4})$

☒ $T(n) = \Theta(n^2)$

☐ $T(n) = \Theta(n^2 \log n)$

☐ $T(n) = \Theta(n^3)$

☐ Rekursionsligningen kan ikke løses med Master Theorem fra Cormen et al., 4. udgave.

Hvilket af nedenstående svar gælder for følgende rekursionsligning?

$$T(n) = 4 \cdot T(n/2) + n^2$$

☐ $T(n) = \Theta(1)$

☐ $T(n) = \Theta(\log n)$

☐ $T(n) = \Theta(n^{\log_4 3})$

☐ $T(n) = \Theta(n)$

☐ $T(n) = \Theta(n \log n)$

☐ $T(n) = \Theta(n^{\log_3 4})$

☐ $T(n) = \Theta(n^2)$

☒ $T(n) = \Theta(n^2 \log n)$

☐ $T(n) = \Theta(n^3)$

☐ Rekursionsligningen kan ikke løses med Master Theorem fra Cormen et al., 4. udgave.

Hvilket af nedenstående svar gælder for følgende rekursionsligning?

$$T(n) = 4 \cdot T(n/3) + n$$

☐ $T(n) = \Theta(1)$

☐ $T(n) = \Theta(\log n)$

☐ $T(n) = \Theta(n^{\log_4 3})$

☐ $T(n) = \Theta(n)$

☐ $T(n) = \Theta(n \log n)$

☒ $T(n) = \Theta(n^{\log_3 4})$

☐ $T(n) = \Theta(n^2)$

☐ $T(n) = \Theta(n^2 \log n)$

☐ $T(n) = \Theta(n^3)$

☐ Rekursionsligningen kan ikke løses med Master Theorem fra Cormen et al., 4. udgave.

Hvilket af nedenstående svar gælder for følgende rekursionsligning?

$$T(n) = T(n/4) + 1$$

☐ $T(n) = \Theta(1)$

☐ $T(n) = \Theta(\log n)$

☐ $T(n) = \Theta(n^{\log_4 3})$

☒ $T(n) = \Theta(n)$

☐ $T(n) = \Theta(n \log n)$

☐ $T(n) = \Theta(n^{\log_3 4})$

☐ $T(n) = \Theta(n^2)$

☐ $T(n) = \Theta(n^2 \log n)$

☐ $T(n) = \Theta(n^3)$

☐ Rekursionsligningen kan ikke løses med Master Theorem fra Cormen et al., 4. udgave.

Hvilke af nedenstående udsagn er sande? [*Et eller flere korrekte svar.*]

	Sand	Falsk
$3n^4$ er $O(4n^3)$	☐	☒
$4n$ er $O(n/3)$	☒	☐
$\sqrt{n}$ er $O(\log n)$	☐	☒
$n^2/2 + n^3/3$ er $O(2n)$	☐	☒
$n \log n$ er $O(n^2/(\log n)^2)$	☒	☐
n^{10} er $O(10^n)$	☒	☐
$\log_4 n$ er $O(\log_3 n)$	☐	☒

Hvilke af nedenstående udsagn er sande? [*Et eller flere korrekte svar.*]

	Sand	Falsk
2^n er $\Omega(n^4)$	☒	☐
n^3 er $o(n^6/2)$	☒	☐
n^2 er $\omega(n^2)$	☐	☒
$n^3/(\log n)^3$ er $\Theta(n^2/(\log n)^2)$	☐	☒
$\log n$ er $\Omega(1)$	☒	☐
2 er $o(1)$	☐	☒
$n - 100$ er $\Theta(n + 100)$	☒	☐

Byg en min-heap ved at udføre BUILD-MIN-HEAP($A,8$) på nedenstående array A :

	1	2	3	4	5	6	7	8
A :	5	4	2	3	7	6	1	0

På hvilken plads i A befinder nøglen 5 sig bagefter?

☐ 1

☒ 2

☐ 3

☐ 4

☐ 5

☐ 6

☐ 7

☐ 8

Et array A

	1	2	3	4	5	6
A :						

indeholder følgende seks nøgler i en eller anden rækkefølge:

$\{5, 6, 7, 8, 9, 10\}$

Det oplyses, at A er en min-heap. Angiv alle pladser i A , som kan indeholde nøglen 9. [*Et eller flere korrekte svar.*]

	Korrekt	Forkert
1	☐	☒
2	☒	☐
3	☒	☐
4	☐	☒
5	☐	☒
6	☐	☒

For en funktion $h_1(x)$ gælder:

$$\begin{aligned} h_1(44) &= 2 \\ h_1(55) &= 5 \\ h_1(66) &= 1 \\ h_1(77) &= 2 \\ h_1(88) &= 5 \\ h_1(99) &= 6 \end{aligned}$$

Vi ser på en hashtabel H af længde syv, der bruger linear probing med ovennævnte funktion $h_1(x)$ som auxiliary hashfunktion. Startende med en tom hashtabel er nøglerne $\{44, 55, 66, 77, 88, 99\}$ blevet indsat i en eller anden rækkefølge. Resultatet er blevet:

	0	1	2	3	4	5	6
H :	99	66	44	77		88	55

Hvilke af nøglerne kan være blevet indsat sidst (dvs. kan have stået sidst i indsættelsesrækkefølgen)? [*Et eller flere korrekte svar.*]

	Korrekt	Forkert
44	☐	☒
55	☒	☐
66	☒	☐
77	☒	☐
88	☐	☒
99	☒	☐

Vi ser på en hashtabel H af længde syv, der bruger double hashing og auxiliary hashfunktioner $h_1(x)$ og $h_2(x)$. Hashtabellen ser lige nu sådan ud:

	0	1	2	3	4	5	6
H :	47	50			35	21	

Derefter indsættes nøglen 99, hvorefter hashtabellen ser sådan ud:

	0	1	2	3	4	5	6
H :	47	50	99		35	21	

Hvis $h_2(99) = 2$, hvilke af følgende værdier af $h_1(99)$ er da mulige? [*Et eller flere korrekte svar.*]

	Korrekt	Forkert
$h_1(99) = 0$	☒	☐
$h_1(99) = 1$	☐	☒
$h_1(99) = 2$	☒	☐
$h_1(99) = 3$	☐	☒
$h_1(99) = 4$	☐	☒
$h_1(99) = 5$	☐	☒
$h_1(99) = 6$	☐	☒

Udfør PARTITION($A,1,9$) på nedenstående array.

	1	2	3	4	5	6	7	8	9
A:	8	5	6	1	9	7	3	2	4

Hvilket af nedenstående svar angiver udseendet af A bagefter?



	1	2	3	4	5	6	7	8	9
A:	1	3	2	4	5	6	9	7	8



	1	2	3	4	5	6	7	8	9
A:	1	3	2	8	9	7	5	6	4



	1	2	3	4	5	6	7	8	9
A:	1	3	2	4	8	5	6	9	7



	1	2	3	4	5	6	7	8	9
A:	1	3	2	4	9	7	5	6	8



	1	2	3	4	5	6	7	8	9
A:	4	2	3	1	9	7	6	5	8



	1	2	3	4	5	6	7	8	9
A:	1	2	3	4	5	6	7	8	9

Et array A af længde ni indeholder tallene $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ i en eller anden rækkefølge. Efter udførelse af $\text{PARTITION}(A, 1, 9)$ ser A sådan ud:

	1	2	3	4	5	6	7	8	9
A :	2	1	3	4	6	5	8	7	9

Hvilke af nedenstående tal kan have været pivot-elementet under PARTITION , dvs. kan have stået på sidste plads i A ved algoritmens start? [*Et eller flere korrekte svar.*]

	Korrekt	Forkert
1	☐	☒
2	☐	☒
3	☒	☐
4	☒	☐
5	☐	☒
6	☐	☒
7	☐	☒
8	☐	☒
9	☒	☐

En fil indeholder nedenstående tegn med de angivne hyppigheder.

Tegn	o	p	q	r	s	t	u
Hyppighed	150	100	25	125	200	50	75

Lav et Huffman-træ på dette input. Hvor mange bits er der i kodeordet for q?

☐ 1

☐ 2

☐ 3

☒ 4

☐ 5

☐ 6

Vi ser stadig på et Huffman-træ for en fil med nedenstående hyppigheder.

Tegn	o	p	q	r	s	t	u
Hyppighed	150	100	25	125	200	50	75

Der er $150 + 100 + 25 + 125 + 200 + 50 + 75 = 725$ tegn i filen. Hvor mange bits fylder disse 725 tegn i alt hvis de kodes med Huffman-koder?

☐ 1800

☐ 1825

☒ 1900

☐ 1925

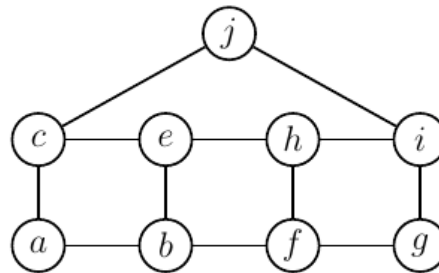
☐ 2000

☐ 2075

☐ 2100

Udfør bredde-først søgning $\text{BFS}(G, a)$ på grafen G nedenfor med start i knuden a .

For BFS afhænger udførelsen af ordningen af knuders nabolister. Du skal i dette eksamenssæt antage, at en knudes naboliste er sorteret i alfabetisk orden efter naboknudernes navne.



Hvilken knude er den *sidste*, som får tildelt d -værdien 3?

☐ e

☐ f

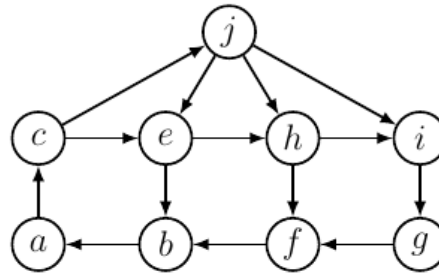
☐ g

☐ h

☒ i

☐ j

Udfør dybde-først søgning $\text{DFS-VISIT}(G, a)$ på grafen G nedenfor med start i knuden a . Læs også næste spørgsmål, inden at du udfører algoritmen. For DFS afhænger udførelsen af ordningen af knuders nabolister. Du skal i dette eksamenssæt antage, at en knudes naboliste er sorteret i alfabetisk orden efter naboknudernes navne.



Hvilken knude er den sidste, som opdages (dvs. som får den højeste discovery time)?

☐ b

☐ c

☐ d

☐ e

☐ f

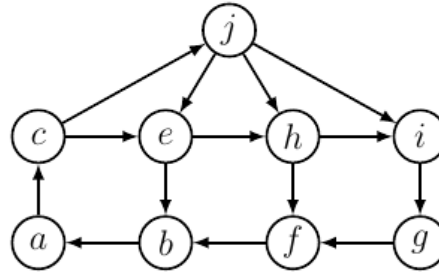
☐ g

☐ h

☐ i

☒ j

Vi ser stadig på dybde-først søgning $\text{DFS-VISIT}(G, a)$ på grafen G nedenfor med start i knuden a (jvf. foregående spørgsmål).



Hvor mange cross edges findes i grafen under denne søgning?



0



1



2



3



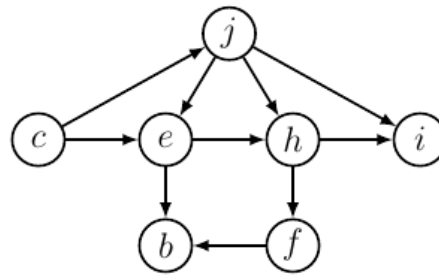
4



5



6



Hvilke af nedenstående lister angiver en topologisk sortering af grafen ovenfor? [Et eller flere korrekte svar.]

	Korrekt	Forkert
c, j, e, h, i, f, b	☒	☐
c, e, h, i, f, b, j	☐	☒
b, c, e, f, h, i, j	☐	☒
j, e, h, i, f, b, c	☐	☒
c, j, e, h, f, i, b	☒	☐
c, j, e, h, i, b, f	☐	☒

Nedenfor er et array af elleve heltal med værdier mellem 0 og 5 (inklusive).

	1	2	3	4	5	6	7	8	9	10	11
A:	2	2	5	1	3	5	2	3	2	5	2

Vi sorterer nu tallene ved at udføre COUNTING-SORT($A, 11, 5$).

I algoritmen bruges et array C med seks pladser (indekseret fra 0 til 5). Hver plads indeholder en tæller, dvs. et heltal.

Hvad er summen af disse seks heltal i C ved algoritmens afslutning?

☐ 0

☐ 5

☒ 11

☐ 18

☐ 23

☐ 32

☐ 34

Vi ønsker at bruge $\text{RADIX-SORT}(A,4)$ til at sortere de seks heltal i nedenstående array A i stigende orden. [Notation fra Cormen et al., 4. udgave: $\text{RADIX-SORT}(A,6,4)$.]

	1	2	3	4	5	6
A:	0113	4102	4440	2240	2213	2340

Hvad er indholdet af indholdet af A efter udførelsen af *to* af de fire iterationer i $\text{RADIX-SORT}(A,4)$?



	1	2	3	4	5	6
A:	4102	0113	2213	4440	2240	2340



	1	2	3	4	5	6
A:	4102	0113	2213	2240	2340	4440



	1	2	3	4	5	6
A:	0113	2240	2213	2340	4102	4440

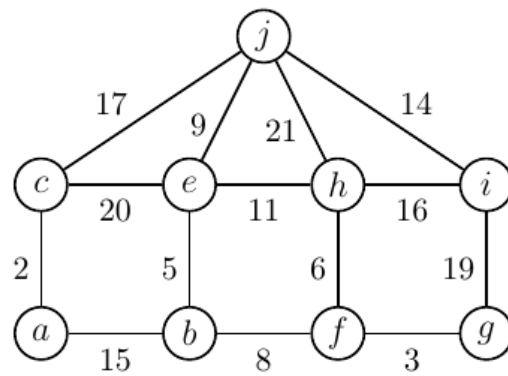


	1	2	3	4	5	6
A:	0113	2213	2240	2340	4102	4440



	1	2	3	4	5	6
A:	4440	2240	2340	4102	0113	2213

Brug Kruskals algoritme til at finde et minimum udspændende træ (MST) for nedenstående graf. Læs også næste spørgsmål, inden at du udfører algoritmen.



Hvilken kant er den sidste, som kommer med i MST?

☒ (a, b)

☐ (i, j)

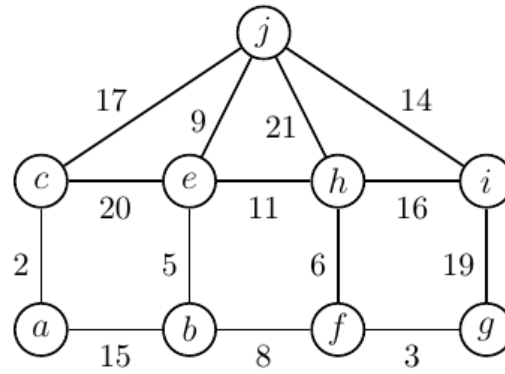
☐ (h, j)

☐ (e, h)

☐ (h, i)

☐ (e, j)

Vi ser stadig på Kruskals algoritme brugt på nedenstående graf (jvf. foregående spørgsmål).



Vi ser på det første tidspunkt i algoritmen, hvor en undersøgt kant ikke tages med i MST.

Hvor mange sammenhængskomponenter har grafen (V, A) på det tidspunkt? [Her er V knuderne i grafen G , og A er de kanter, som indtil nu er taget med i MST.]

☐ 1

☐ 2

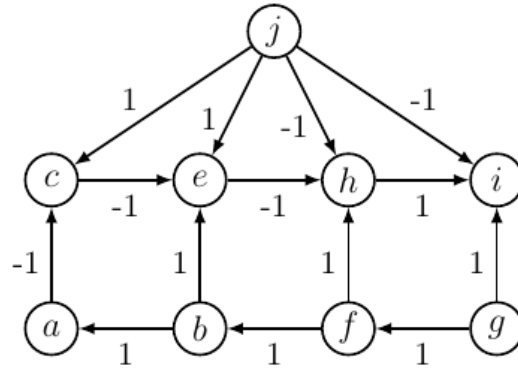
☐ 3

☐ 4

☒ 5

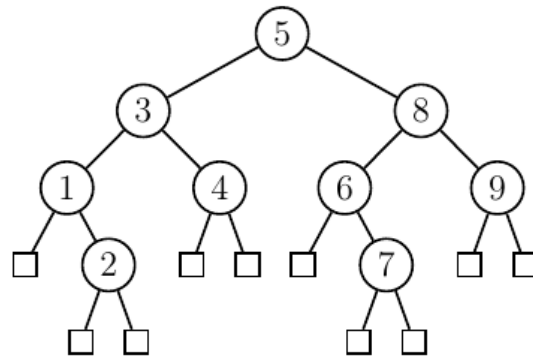
☐ 6

Hvilke af disse algoritmer kan bruges til at finde korteste veje på nedenstående graf? *[Et eller flere korrekte svar.]*



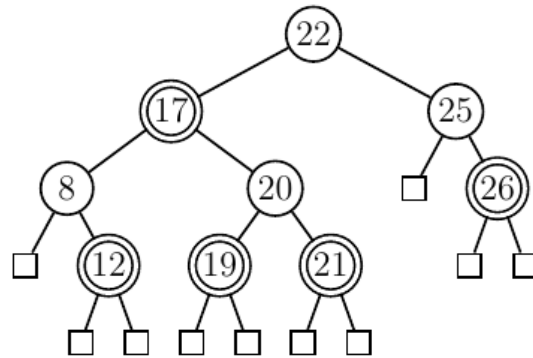
	Korrekt	Forkert
DIJKSTRA	☐	☒
BELLMAN-FORD	☒	☐
DAG-SHORTEST-PATHS	☒	☐
KRUSKAL	☐	☒
BREADTH-FIRST-SEARCH	☐	☒
DEPTH-FIRST-SEARCH	☐	☒
FLOYD-WARSHALL	☒	☐
PRIM	☐	☒

Hvilke af nedenstående delmængder af knuder vil gøre træet til et lovligt rød-sort træ, hvis netop denne delmængde af knuder farves røde (og resten farves sorte). [*Et eller flere korrekte svar.*]



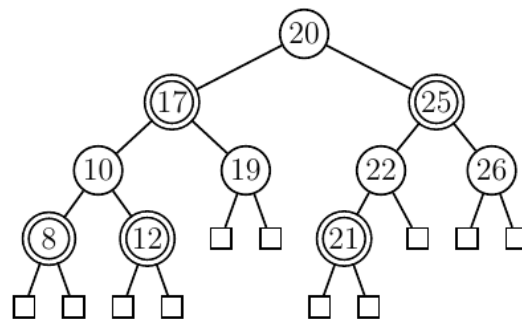
	Korrekt	Forkert
2, 7	☒	☐
1, 2, 4, 7, 8	☐	☒
2, 5, 7	☐	☒
2, 3, 6, 7, 9	☐	☒
2, 3, 7, 8	☒	☐

Indsæt nøglen 10 i nedenstående rød-sorter træ (dobbeltcirkler angiver røde knuder).

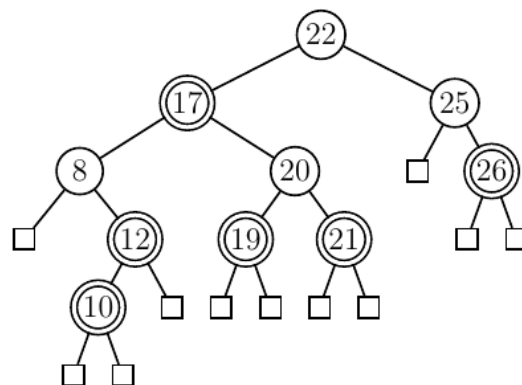


Hvilket af følgende svar viser træet efter indsættelsen?

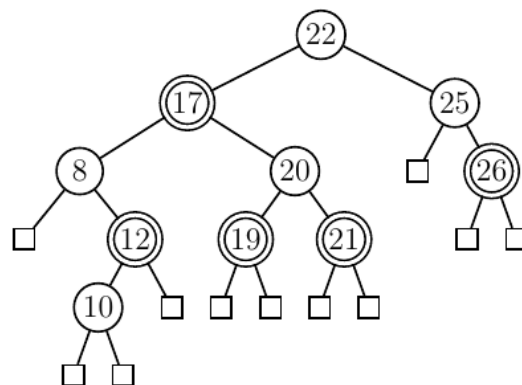
☐

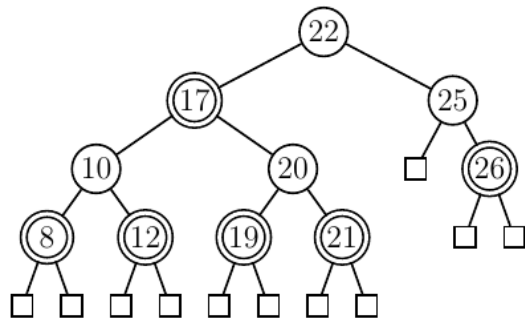


☐



☐



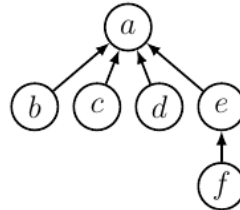
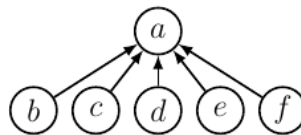
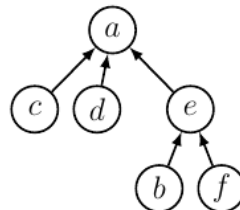


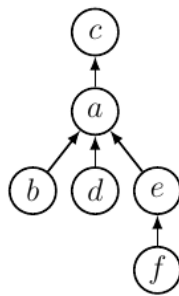
I denne opgave ser vi på den træ-baserede datastruktur for disjoint-sets (også kaldet disjoint-set forest). De to heuristikker union-by-rank og path compression anvendes begge. Som sædvanligt refererer metodenavne til pseudo-koden i Cormen et al., 4. udgave.

Startende med en tom struktur laves denne sekvens af operationer:

```
MAKE-SET( $a$ )
MAKE-SET( $b$ )
MAKE-SET( $c$ )
MAKE-SET( $d$ )
MAKE-SET( $e$ )
MAKE-SET( $f$ )
UNION( $f$ ,  $e$ )
UNION( $b$ ,  $f$ )
UNION( $d$ ,  $a$ )
UNION( $f$ ,  $d$ )
UNION( $b$ ,  $c$ )
FIND-SET( $a$ )
```

Hvilken af følgende figurer angiver datastrukturen efter denne sekvens?

☐

☐

☐




2 point

For følgende algoritme, hvad er den asymptotiske køretid i Θ -notation som funktion af n ?

```
ALGORITME1( $n$ )  
   $s = 0$   
  while  $s < n$   
     $s = s + 3$ 
```

☐ $\Theta(\log n)$

☐ $\Theta(\sqrt{n})$

☒ $\Theta(n)$

☐ $\Theta(n \log n)$

☐ $\Theta(n^2)$

☐ $\Theta(n^3)$

For følgende algoritme, hvad er den asymptotiske køretid i Θ -notation som funktion af n ?

```
ALGORITME2( $n$ )  
   $s = 0$   
   $i = 0$   
  while  $i < n/2$   
    for  $j = i$  to  $i + n/2$   
       $s = s + 1$   
     $i = i + 1$ 
```

- ☐ $\Theta(\log n)$
 - ☐ $\Theta(\sqrt{n})$
 - ☐ $\Theta(n)$
 - ☐ $\Theta(n \log n)$
 - ☐ $\Theta(n^2)$
 - ☐ $\Theta(n^3)$
-

For følgende algoritme, hvad er den asymptotiske køretid i Θ -notation som funktion af n ?

```
ALGORITME3( $n$ )  
   $i = 1$   
  while  $i \leq n$   
     $j = 1$   
    while  $j \leq n$   
       $j = j * 2$   
     $i = i + 2$ 
```

☐ $\Theta(\log n)$

☐ $\Theta(\sqrt{n})$

☐ $\Theta(n)$

☐ $\Theta(n \log n)$

☒ $\Theta(n^2)$

☐ $\Theta(n^3)$

For følgende algoritme, hvad er den asymptotiske køretid i Θ -notation som funktion af n ?

```
ALGORITME4( $n$ )  
   $i = 1$   
   $j = n$   
  while  $i \leq j$   
     $i = i * 2$   
     $j = j/2$ 
```

☐ $\Theta(\log n)$

☐ $\Theta(\sqrt{n})$

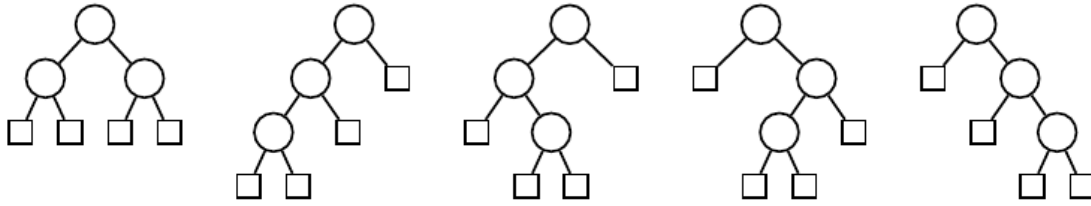
☐ $\Theta(n)$

☐ $\Theta(n \log n)$

☒ $\Theta(n^2)$

☐ $\Theta(n^3)$

I denne opgave ønsker vi at finde antallet $B(n)$ af binære træer med n knuder. Eksempelvis er $B(3) = 5$, eftersom der findes fem binære træer med tre knuder:



Baseret på en case-analyse af størrelsen af rodens to undertræer kan man vise følgende rekursionsligning for $B(n)$:

$$B(n) = \begin{cases} 1 & \text{hvis } n = 0 \\ \sum_{i=0}^{n-1} B(i) \cdot B(n-i-1) & \text{hvis } n > 0 \end{cases}$$

Hvis $B(n)$ findes via dynamisk programmering baseret på denne rekursionsformel, hvilken køretid opnås da?

- ☐ $\Theta(1)$
 - ☐ $\Theta(\log(n))$
 - ☐ $\Theta(n)$
 - ☐ $\Theta(n \log n)$
 - ☐ $\Theta(n^2)$
 - ☐ $\Theta(n^3)$
-

Vi ser stadig på at bestemme antallet $B(n)$ af binære træer med n knuder ved hjælp af nedenstående rekursionsformel.

$$B(n) = \begin{cases} 1 & \text{hvis } n = 0 \\ \sum_{i=0}^{n-1} B(i) \cdot B(n-i-1) & \text{hvis } n > 0 \end{cases}$$

Hvis $B(n)$ findes via dynamisk programmering baseret på denne rekursionsformel, hvad er det mindste pladsforbrug, som kan opnås?

☐ $\Theta(1)$

☐ $\Theta(\log(n))$

☐ $\Theta(n)$

☐ $\Theta(n \log n)$

☐ $\Theta(n^2)$

☐ $\Theta(n^3)$